Mp10 Web — Installation

Installing and updating the browser application

Structured Systems · www.structuredsystems.com

03 November 2026

Contents

1 About this guide **3**

2 Prerequisites **4**

3 Before you install: verify the download **5**

4 Installing **6**
4.1 If PowerShell refuses to run it . . . 6
4.2 The questions . . . 7
4.3 Remembered answers . . . 8
4.4 Accounts: this installer creates none, on purpose . . . 8
4.4.1 What must exist before the first sign-in, and why . . . 9
4.4.2 Which group sees what . . . 9
4.4.3 A forgotten password *(IT task)* . . . 10
4.5 When it finishes . . . 10

5 The workstation side, which this installer does not do **12**
5.1 There is a second installer for this . . . 12
5.2 The one thing that connects the two halves . . . 12

6 What it touches **14**
6.1 What the bundle contains . . . 14
6.2 How it mounts into your Apache . . . 14

7 Electronic claim submission needs AutoTasks **16**

8 TLS **17**

9 Updating **18**

10 Removing it **19**

11 Fault-finding **20**
11.1 Step 0/8 — Privileges . . . 20
11.2 Step 1/8 — Verify the bundle . . . 20
11.3 Step 2/8 — Preflight . . . 21
11.4 Step 3/8 — Configuration . . . 22
11.5 Step 4/8 — Test the dictionary connection . . . 23
11.6 Step 5/8 — Deploy files . . . 23
11.7 Step 7/8 — Apache . . . 23
11.8 Step 8/8 — Database . . . 24
11.9 Smoke test . . . 24

1 About this guide

Mp10 Web is the browser-based front end for patients, encounters, insurance, claims, remittances, electronic claim submission and reports — a PHP 8 API and a Vue 3 single-page app, reading and writing the same Advantage Database Server dictionary (`mp.add`) the Mp10 Win32 desktop applications already use.

This guide is for whoever installs and maintains it on a server. It covers one thing: taking a published bundle (a single zip) and turning it into a running site with `install.ps1`, the script the bundle ships. Everything here is **IT task** — there is nothing in this guide for clinic staff.

If you are setting up a development copy from source instead of a customer site, see `Web/INSTALL.md` in the repository — the manual, step-by-step path remains there. This guide is the server-install path: one script, a handful of questions, and a running site inside the Apache the server already has.

There are three installers. This guide covers the server one. A second provides PHP and Apache for a server that has neither; a third goes to each desk that prints, signs or scans.

Bundle	Runs on	Installs	When
`mp10-prereqs-<version>.zip`	the server	PHP 8.x ZTS x64 with `php_ads`, the 64-bit ACE client, Apache + `mod_fcgid`	only if the server lacks them
`Mp10Web-Server-Installer.zip`	the server	the web application, into that server's Apache	every install and update
`Mp10Web-Workstation-Installer.zip`	each workstation	MpPrintSrv and MpSigSrv, pointed at the site	each desk that prints, signs or scans

Run them in that order. The prerequisites installer ends by printing the exact `-PhpPath` and `-ApacheService` arguments the next one wants, so its answers are already in hand.

2 Prerequisites

The bundle carries the application only. PHP, Apache and the ADS client belong to the server, and this installer can only diagnose a wrong one, not fix it — so get these right first.

> **A server that has none of this has its own installer.** Extract `mp10-prereqs-<version>.zip` and run `Install-Prerequisites.ps1` elevated: it surveys what is already there, installs only what is missing, and refuses rather than laying a second PHP beside an unsuitable one. It carries everything it needs, so the server requires no internet access. What it cannot provide is the Advantage Database Server itself — SAP's licensed product, with no public download.

The requirements themselves:

- **PHP 8.x, the ZTS (thread-safe) build, x64, with `php_ads` loaded.** Windows PHP ships four builds (NTS/ZTS × x86/x64); this needs the *Thread Safe* x64 zip from windows.php.net, with `extension=ads` enabled in `php.ini` and `php_ads.dll` in the extension directory.
- **The 64-bit ACE client on `PATH`** — `ace64.dll`, `adsloc64.dll`, `aicu64.dll`, `axcws64.dll`. **The Mp10 desktop applications install the 32-bit ACE client.** That is a different, separate install: it satisfies the desktop apps and will not satisfy this PHP. The installer's preflight check exists specifically to catch this and says so by name.
- **`php-cgi.exe` next to `php.exe`.** Mp10 Web serves PHP through `mod_fcgid`, which needs the CGI build alongside the CLI one; a PHP zip normally ships both together, but confirm it.
- **Apache 2.4, installed and registered as a Windows service**, a full build such as Apache Lounge's, with `mod_rewrite.so` and `mod_fcgid.so` present on disk under `<Apache root>\modules\`. Mp10 Web is served **by that Apache**, at an address such as `https://server.clinic.local/mpweb` — see "How it mounts into your Apache" below. It has to be a *service* because that is how the installer finds it, and how it restarts it afterwards; an Apache started by hand from a command prompt is not discovered. A trimmed build missing `mod_fcgid.so` will not do.
- **TLS, if the site wants it, is that Apache's own.** Mp10 Web adds no listener, no port and no certificate — it appears on whatever the server already serves. A site terminating HTTPS today serves Mp10 Web over HTTPS with no extra work; one serving plain HTTP serves Mp10 Web the same way.

3 Before you install: verify the download

The bundle publishes as two files: a zip (`Mp10Web-Server-Installer.zip`) and a `.sha256` sidecar beside it. `manifest.json` inside the zip verifies every file *once extracted*, but it cannot verify itself, or the zip that carries it — that is what the sidecar is for, and it has to be checked **before** you extract, not after:

```
Get-FileHash .\Mp10Web-Server-Installer.zip -Algorithm SHA256
```

Compare the `Hash` value against the contents of `Mp10Web-Server-Installer.zip.sha256` (a plain text file: the hash, two spaces, the filename). If they do not match, download again — do not extract a zip whose checksum does not match its sidecar, even if it looks fine.

Once that checks out, extract the zip. `manifest.json` and `install.ps1` land beside each other at the top of the extracted folder — that is the "bundle" the rest of this guide refers to.

4 Installing

From an **elevated** PowerShell, in the extracted bundle folder:

```
.\install.ps1
```

Everything it needs, it asks for, so the bare command above is the normal way to run it.

4.1 If PowerShell refuses to run it

The script is not code-signed, so on a server whose execution policy is `AllSigned` or `Restricted` the bare command stops with *"is not digitally signed"* or *"running scripts is disabled on this system"*. Run it through a one-process bypass instead — this changes nothing in the machine's policy; it applies to that single invocation only:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\install.ps1
```

Parameters go after the script name exactly as before, and for an update `-NonInteractive` can be added at the end.

If it still refuses, or every file prompts *"Do you want to run this software?"*, the extracted files carry the mark-of-the-web from the download. Clear it once, from the bundle folder, then run the command above again:

```
Get-ChildItem -Recurse | Unblock-File
```

Only the bundle folder is touched; nothing under the install root or the site's Apache is changed by either command.

Every answer can also be given as a parameter, which then skips that prompt:

Parameter	What it decides
`-InstallRoot <path>`	Where Mp10 Web is deployed. Default `C:\Mp10Web`.
`-PhpPath <path to php.exe>`	Which PHP serves it. Default: `php.exe` on `PATH`, else `C:\php\php.exe`.
`-ApacheService <name>`	Which Apache **service** it mounts into.
`-HttpdConf <path>`	The `httpd.conf` that service runs, if it is not the one derived from the service itself.
`-UrlPrefix <path>`	The URL path, e.g. `/mpweb`.
`-PublicOrigin <scheme://host[:port]>`	The address staff type.
`-DdPath` / `-AdsMode` / `-AdsUser`	The dictionary connection.
`-AnswersFile <path>`	Where remembered answers live. Default `%ProgramData%\Mp10Web\install-answers.json`.
`-NonInteractive`	Never prompt; use defaults and stop on anything that has none.
`-AdsPassword <password>`	For an unattended **first** install only — see the warning under "Remembered answers".

On a machine with more than one Apache service, say which one. The installer lists every `httpd.exe` service it can find, with the configuration each one runs, and offers a default:

```
Apache services on this machine:
   Apache2.4                  Running   C:\Apache24\conf\httpd.conf
   SomeOtherApp-httpd         Running   D:\OtherApp\apache\conf\httpd.conf
Apache service to mount Mp10 Web into [Apache2.4]:
```

Getting this wrong is not subtle — it means the Include lands in a different site's configuration — so read the list rather than pressing Enter through it on a server you do not know.

The script runs eight steps plus a smoke test, in an order deliberately chosen so that **nothing is written until every check that can run has run** — a half-configured install is worse than a refused one:

Step	What happens
0/8 Privileges	Confirms you are elevated.
1/8 Verify the bundle	Hashes every file in `manifest.json` against what is actually on disk.
2/8 Preflight	Checks PHP's version, bitness, thread-safety and the `ads` extension; finds the Apache service, its `httpd.conf` and its module files.
3/8 Configuration	Asks the questions below.
4/8 Test the dictionary connection	Connects to `mp.add` with what you just typed, **before writing anything**.
5/8 Deploy files	Copies the application into the install root, and sets the SPA's `<base href>` to the URL path.
6/8 Configuration file	Writes `database.local.php` (fresh install only — see "Updating").
7/8 Apache	Renders `<install root>\apache\mp10web.conf`, adds one `Include` line to the site's `httpd.conf`, validates it with `httpd -t`, and restarts Apache only if something changed.
8/8 Database	Runs the migration: creates the `web_users`, `web_groups`, `web_group_members` and `web_x12jobs` **tables** if they don't exist. It creates no accounts and grants no permissions — see "Accounts" below — and reports how many active accounts the dictionary already has.
Smoke test	Posts a login for a random username through the real URL and requires a 401 back.

4.2 The questions

Asked once, interactively (under `-NonInteractive` each falls back to its default instead):

1. `php.exe` — the PHP that will serve the site.
2. **Apache service** — which Apache instance to mount into (see above).
3. `httpd.conf` — the configuration that service runs; derived from the service itself, and normally just confirmed with Enter.
4. **Install root** — where the application is deployed. Default `C:\Mp10Web`.
5. **ADS mode — Remote or Local.** Remote is normal: the dictionary lives on an ADS server elsewhere on the network.
6. **Path to `mp.add`.** The existing dictionary — remote paths look like `\\HOST:6262\VOLUME\path\mp.add`.
7. **ADS username.** Defaults to `adssys`.
8. **ADS password** (typed hidden). On an update, Enter keeps the stored one.
9. **URL path.** Where Mp10 Web appears on this server — default `/mpweb`, giving `https://<host>/mpweb/`. It cannot be bare `/`: the site's own document root belongs to whatever that Apache already serves.

Answer it with the exact capitalisation staff will type. Apache matches an `Alias` case-sensitively, so a site installed at `/MpWeb` serves `https://host/MpWeb/` and answers **404** to `https://host/mpweb/`. Verified, not assumed.

10. **Public origin.** The exact address staff will type into their browser — scheme and host, no path. It sets `cors_origins` in `database.local.php`, and it is the value each workstation's print and signature helper must be given. The default is derived from this machine's name and the ports its Apache listens on.

4.3 Remembered answers

Every successful run writes what it was told to `%ProgramData%\Mp10Web\install-answers.json`, and the next run offers those values as the default for each prompt. Rolling out an update is then a matter of pressing Enter through the questions, and a scripted update needs no arguments at all beyond `-NonInteractive`.

Precedence, when the same answer has more than one possible source: a **parameter** on the command line wins, then the **answers file**, then a value read back from the **existing install**, then the built-in default.

```
{
    "schema": 1,
    "written_utc": "2026-08-18T15:26:01Z",
    "install_root": "C:\\Mp10Web",
    "php_path": "C:\\php\\php.exe",
    "apache_service": "Apache2.4",
    "httpd_conf": "C:\\Apache24\\conf\\httpd.conf",
    "ads_mode": "Remote",
    "dd_path": "\\\\HOST:6262\\VOL\\dictionary\\mp.add",
    "ads_user": "adssys",
    "url_prefix": "/mpweb",
    "public_origin": "https://mp10.clinic.local"
}
```

The ADS password is never written there — not encrypted, not hashed, not "just the first part". On an update the installer reads it back from the site's own `database.local.php` instead, which is where it already lives. The one case with neither source is an unattended *first* install, and `-AdsPassword` exists for exactly that; prefer running the first install interactively, because a password on a command line is readable by anything that can see the process's arguments and lands in the shell's history.

A saved answer is a **default, not a command**: an interactive run still shows every question with the remembered value pre-filled, so a site that has moved its dictionary can simply type the new path over the old one.

4.4 Accounts: this installer creates none, on purpose

The installer creates tables, never users. Step 8/8 makes `web_users`, `web_groups`, `web_group_members` and `web_x12jobs` exist; it puts nothing in them. A freshly installed site therefore has **no one who can sign in**, and the installer says so plainly rather than letting you find out at the login screen:

```
[WARN] No active MpWeb accounts exist in this dictionary - nobody can sign in yet.
[WARN] Create the first one in the desktop Admin module (see the summary below).
```

That is a deliberate change from how this used to work. The installer once seeded an `admin` account and printed a generated password once. That left every site carrying the same kind of standing shared credential, recoverable only by running a script on the server — and a seeded permission list that drifted from what the API actually enforces. **There is now no default login to forget to change.**

4.4.1 What must exist before the first sign-in, and why

All three are done in the **desktop Admin module**, which already owns the dictionary users and groups that MpWeb's permissions key on:

#	What	Where	Why it is needed
1	**MpWeb permissions on each dictionary group** — the `web_groups` rows	Admin → structure update	MpWeb decides what a user may do by matching their dictionary groups against `web_groups.group_name` and unioning that row's `perms`. With no row, a user signs in with **no permissions** — which reads as a broken application, not a missing setting. The update is additive and re-runs safely, so permissions added in a later release reach the site the same way.
2	**An MpWeb account for the operator** — the `web_users` row	Admin → Users screen	It holds the username, the bcrypt password hash and `is_active`. This is the credential itself; nothing else creates one.
3	**That operator's membership of a dictionary group**	Admin → Users screen	Membership **is** the authorisation. MpWeb resolves it from the dictionary at every sign-in, so changing a group in Admin takes effect the next time the person signs in — nothing to re-run here.

The join that makes it work is `web_users.operator` = the dictionary user name, exactly. Admin never asks for that code as free text — it uses the user name the Users screen is already editing — because a mismatch produces an account that signs in successfully and can then do nothing at all.

Order matters only between 1 and 2/3: do the structure update first, or the first person to sign in will have an account with no capabilities.

4.4.2 Which group sees what

The structure update writes one `web_groups` row per dictionary group below. A person's menu is the union of the rows for the groups they belong to, so **the group you put an operator in decides which screens exist for them**. There is no separate "web role"; there is only dictionary group membership.

Dictionary group	Sees in Mp10 Web
Administrators	Patients and Encounters (edit), Claims (view and edit), Reports, the Modality worklist, and the Admin screens for the print and signature helpers
Admission	Patients and Encounters (edit), and an encounter's claims list read-only. **No** Claims page, Remittances, Submissions or Reports
BillingUsers	Patients and Encounters (read-only), Claims (view, edit, raise, print, payments), Remittances including **Post**, Submissions (Create 837 and Send), Reports
AdjustmentAuthUsers	Only the right to enter a non-zero adjustment on a payment line. Opens nothing on its own — it is granted **in addition to** BillingUsers, matching how the desktop authorises adjustments
Transcription	The Modality worklist only

Anything not in that table gets no row and its members sign in to the Dashboard and nothing else.

Three consequences worth telling the site before the first sign-in, because each one looks like a fault and is not:

- **Not every operator is a billing user.** An admissions clerk does not see Claims, Remittances, Submissions or Reports, and should not. When someone reports a screen "missing", check their groups in the Admin module before anything else.
- **Administrators do not post remittances, submit 837s or adjust balances.** Those three are money actions and are granted to the billing groups only, exactly as the desktop does. An administrator who also bills is a member of BillingUsers as well.
- **Group names are matched exactly** as the Admin module creates them (`Admission`, singular). A group added by hand under a near-miss name writes happily and matches nobody.

A site that wants a different split (for instance, posting restricted to a supervisor group) changes the declaration in the Admin module, not the web app, and reconciles; the update is additive, so a permission removed there also has to be removed from the existing row by hand.

4.4.3 A forgotten password *(IT task)*

Open Admin → Users and set a new one. There is no script to run on the server and nothing to reset in the dictionary by hand — which is the point of moving provisioning there. Disabling an account clears `is_active`, which blocks sign-in while leaving the operator code meaningful on every row that account has already written; accounts are not deleted.

4.5 When it finishes

A successful run ends with a summary:

```
Mp10 Web is installed.
  URL          : https://mp10.clinic.local/mpweb/
  Install root : C:\Mp10Web
  Config       : C:\Mp10Web\backend\php\config\database.local.php
  Apache       : service 'Apache2.4', Include in C:\Apache24\conf\httpd.conf -> C:\Mp10Web\apache\mp10w
  httpd.conf   : previous copy at C:\Apache24\conf\httpd.conf.20260818-131709.mp10web.bak

  BEFORE ANYONE CAN SIGN IN - in the desktop Admin module, not here:
    1. Run a structure update, so each dictionary group gets its MpWeb
       permissions (web_groups). Without it a user signs in with none.
```

```
  2. Users screen -> give the operator an MpWeb account and password.
  3. Confirm that operator belongs to a dictionary group - group
     membership IS the authorisation; MpWeb reads it at every sign-in.
This installer deliberately creates no accounts: there is no default
login to forget to change.
```

That three-step block appears **only when the dictionary has no active MpWeb accounts** — on an update of a site already in use it is absent, because nothing is outstanding.

Open the URL. Whatever certificate warning the site shows (or does not show) is the server's existing TLS setup — Mp10 Web neither adds nor changes one.

The site is now running, nobody can sign in until Admin creates an account, and three further things will not work until the workstations are set up. All of that is expected, and none of it is a fault in the install.

5 The workstation side, which this installer does not do

Everything above sets up the **server**. Three features do not live there, because the hardware they need is plugged into the operator's own desk and a browser cannot reach it:

Feature	Needs, on each workstation
Printing encounter forms, labels and results	**MpPrintSrv**
Patient signatures on a Topaz pad	**MpSigSrv**
Scanning documents and cards	**MpSigSrv** (same program)

Each is a small program that runs at the desk and is driven over loopback by the browser. Neither is in this bundle and neither is installed by `install.ps1`. Until they are set up, those buttons are greyed out or report that the helper is not answering — on a site that is otherwise perfectly healthy. **Do not read that as a failed install.**

5.1 There is a second installer for this

The desk half has its own bundle, built alongside this one, which installs **both** helpers at a workstation:

```
Expand-Archive Mp10Web-Workstation-Install.zip -DestinationPath C:\Mp10Helpers
C:\Mp10Helpers\Install-Workstation.ps1
```

Run it elevated, at each desk that prints, signs or scans. It asks for the Mp10 program folder and the origin, copies `MpPrintSrv.exe`, `MpSigSrv.exe`, `FrSystH.dll` and `EZTW32.DLL` into that folder, writes `PORT` and `ORIGIN` under `[PRINT]` and `[SIGN]` in `Mp10.ini` — editing only those lines, apart from adding a commented-out `[RDD]` template when that section is missing entirely — registers MpPrintSrv as a service, and checks both helpers over loopback.

It defaults every answer to what that desk already says, reading the existing `Mp10.ini` first, so a re-run or a change of address is a matter of pressing Enter.

Two things it prints instead of doing, because doing them would be wrong: **MpSigSrv's startup** (a shortcut made by a technician lands in the technician's profile, and the operator's session silently never starts it), and **SigPlus** and the **32-bit ACE client**, which are installed by Topaz's own installer and by the Mp10 desktop applications respectively. It detects both and reports which feature each gap blocks.

The two guides remain the reference for everything else about the helpers, and both are IT tasks:

- **Mp10 Web — Printing** for MpPrintSrv — and it carries the fuller description of the workstation installer.
- **Mp10 — Signature Helper** for MpSigSrv, which despite its name covers scanning as well as the signature pad.

5.2 The one thing that connects the two halves

The public origin you answered must reach every workstation. It is the one value the two installers share, and nothing checks them against each other: this installer writes it into the server's `cors_origins`, and the workstation installer writes the same string into each desk's `Mp10.ini`. Give the workstation installer that exact value — it is what its Origin prompt is asking for.

Each helper reads an allowlist from `Mp10.ini` beside it:

```
[PRINT]
ORIGIN=https://mp10.clinic.local

[SIGN]
ORIGIN=https://mp10.clinic.local
```

Exactly as the browser sends it: scheme and host, the port when it is not 80 or 443, and no trailing slash. It must match the public origin you answered, and it is the ORIGIN alone - never the /mpweb path, which the helpers do not see.

Get this wrong and **every request from the browser is refused** — and a browser cannot tell a refused origin from a helper that is not running. Both arrive as the same message. It is the single most common reason a correctly installed site cannot print or sign, so check it before anything else, and check it on the workstation rather than the server.

Two practical notes that cost time when they are not known:

- **Each helper reads its port and origin once, at startup.** After editing `Mp10.ini`, restart it — `Restart-Service MpPrintSrv` for the print helper; MpSigSrv is **not** a service and is ended and relaunched in the operator's own session.
- **Ask the helper what it believes**, rather than reading the ini and assuming. On the workstation, open `http://127.0.0.1:6265/status` and `http://127.0.0.1:6266/status`. Each reports the origins it will accept. `"origins":[]` means none were configured and every browser request will be refused.

6 What it touches

Deliberately little, and every bit of it is listed here:

- **Four tables in the dictionary**: `web_users`, `web_groups`, `web_group_members`, `web_x12jobs` (the electronic-submission queue). The desktop Admin module declares the same four, so a dictionary that has had a structure update already has them. The clinical schema, its stored procedures, and the `NewSeqKey` key-minting function all belong to the **Win32 Admin module** — this installer neither creates nor inspects them. If a dictionary is missing something Mp10 Web needs at that level, that is fixed by running Admin, not by this installer.
- **Nothing in the `sequences` table.** The migration *reads* that table's `recno` counter and reports whether the next number is already taken — no more. It used to raise the counter to match the highest `patients.recno`; that write was removed on 2026-08-06, because on a measured production restore it would have burned 2.6 million record numbers, and it must not be restored. What keeps record numbers from colliding is the API retrying a taken one, not this counter.
- **One config file**, `database.local.php`, under the install root.
- **One rendered Apache configuration**, `mp10web.conf`, under the install root — never inside the site's own configuration directory.
- **Three lines in the site's `httpd.conf`**: an `Include` between two comment markers. Nothing else in that file is read as configuration or rewritten, and a timestamped `.bak` copy is taken before the edit.
- **One answers file**, `%ProgramData%\Mp10Web\install-answers.json`.

6.1 What the bundle contains

Only what a running site needs: the built SPA, the PHP API and its library, the `web_*` table schema, `tools\migrate.php`, the Apache configuration template, the installer itself, and these two guides — around 70 files.

Three things are deliberately kept out, and an update **removes** them from a site installed with an older bundle that still carries them:

- **The test suite** (`backend\php\tests`). Not clutter — a hazard. Its bootstrap loads that site's own `database.local.php`, and the tests create and delete real patients, users and groups. Shipping it would put `php run.php` one command away from writing to live clinical data on a machine where nobody would expect a test suite to exist.
- **Development tooling** — the bundle builder, its test scripts, and `get-apache.ps1` (a helper for the superseded dedicated-Apache design; following it would set up an Apache this installer no longer uses).
- **Internal documents** — this project's plans and specs, and its known-gaps `TODO.md`. They record how Mp10 Web was built and what is deliberately unfinished; neither is documentation *of* the product.

None of that relies on remembering to exclude anything: the repository keeps build and install tooling in `Web\Installer\` and site tooling in `Web\tools\`, and only the latter is copied. `Web\Installer\test-bundle.ps1` asserts it on every build, with an allowlist for `tools\`, so a helper added later cannot drift into a customer's install unnoticed.

6.2 How it mounts into your Apache

Mp10 Web runs **inside the Apache the server already has**, as an alias. It adds no service, no port and no certificate of its own.

The whole of the edit to the site's `httpd.conf` is this:

```
# BEGIN Mp10 Web (managed by install.ps1 - do not edit between the markers)
Include "C:/Mp10Web/apache/mp10web.conf"
# END Mp10 Web
```

Everything else lives in `C:\Mp10Web\apache\mp10web.conf`, which the installer owns and re-renders on every run:

- **Two aliases** — `/mpweb/api` to the PHP API, and `/mpweb` to the built SPA. The API alias is written first because `mod_alias` takes the first match in file order.
- **`mod_rewrite` and `mod_fcgid`**, each loaded only if the site has not loaded it already (`<IfModule !…>`). `mod_rewrite` is not optional: the API's `.htaccess` uses it to re-attach the `Authorization` header Apache strips from the CGI environment, and without it every signed-in request returns 401 while holding a perfectly valid token.
- **PHP through `mod_fcgid`**, pointed at the server's own `php-cgi.exe` — the same PHP the preflight step validated. The bundle ships no PHP.
- **`DirectorySlash Off` on the API directory.** Route families are real subdirectories, so `POST /mpweb/api/patients` matches a directory; without this, `mod_dir` would answer it with a 301 to `.../patients/` *before* the front-controller rewrite runs, and a browser replays a redirected POST as a GET with no body — "create patient" would silently become a no-op.
- **`Require all denied`** on `config/`, `lib/` and `vendor/`.

Three server-wide `mod_fcgid` settings (`FcgidInitialEnv PHPRC`, `FcgidMaxRequestLen`, `FcgidIOTimeout`) are written **only if the site does not already set them** — and only the first announces itself when it is skipped, so a site whose own `FcgidMaxRequestLen` is smaller than 40 MB will truncate uploads with no hint from this run. That check matters: a second `FcgidInitialEnv PHPRC` would silently repoint every other PHP application that Apache serves at Mp10 Web's `php.ini`. When the installer finds one, it says so and leaves it alone.

Before anything goes live the whole configuration is validated with `httpd -d <server root> -f <httpd.conf> -t`, naming the file explicitly. **If Apache rejects it, the `httpd.conf` backup is restored and the run stops** — the site is never left holding a configuration it would refuse at its next restart. Apache is restarted only when the rendered file or the `Include` actually changed, so a routine update of the application alone does not interrupt anything the server is serving.

7 Electronic claim submission needs AutoTasks

The **Submissions** screen queues work; it transmits nothing itself. Building an 837 file from the queued claims, and later sending it to the clearinghouse, is done by the **AutoTasks** service on its regular sweep — the same service that fetches eligibility, imports 835s and keeps the modality worklist. That is deliberate: the clearinghouse credentials, the file paths and the X12 writer already live in the desktop and the service, and the web app was not given a second copy of any of them.

For submission to work at a site:

AutoTasks is installed and running	against the **same dictionary** the web app uses. See the *Mp10 AutoTasks* guide.
`AUTOTASKS` / `PROCESS_837_JOBS` is `YES`	in `sys_registry`. The default **is** `YES`; a site that bills from the desktop only sets it `NO`, and then nothing queued from the web is ever built.
`X12FILE_PATH` / `837` names a directory **the service's account can write**	the built file is placed there. A path the desktop can reach but the service cannot fails the build, and the job says so.
The clearinghouse settings the desktop already uses are in place	sending reuses them; nothing is configured on the web side.

A job that never leaves *Queued* means the service is not running against this dictionary, or the switch is `NO`. The web page has no view of the service and cannot tell you which; check the service first. From the server, `AutoTasks --job=0 --no-send` builds every queued row from the command line regardless of the switch, printing what it does, which is the quickest way to prove the service side on a new site. Keep the `--no-send`: without it the same command also transmits anything already marked for sending.

When the web bundle is updated, update AutoTasks from the same release. The job protocol between the two is shared, and an older service may not recognise what a newer page queues.

8 TLS

There is nothing to do here, and nothing the installer will do. Mp10 Web appears on whatever the site's Apache already serves: if that is HTTPS with a real certificate, so is Mp10 Web, on the same certificate; if it is plain HTTP, Mp10 Web is served over plain HTTP too.

Two consequences worth being deliberate about:

- **Answer the "Public origin" question with the scheme the site actually serves.** It is written into `cors_origins` and handed to the workstation helpers; `http://` where the server serves `https://` is a mismatch nobody notices until printing or signing quietly stops working.
- **A clinic serving patient data over plain HTTP is a decision, not a default.** Mp10 Web will not make it for you, and it will not warn about it either — it is the server's setting, made before Mp10 Web arrived.

9 Updating

Run the same command against the same install root:

```
.\install.ps1
```

Nothing needs to be given again: the answers file supplies the install root, the Apache service, the URL path and the rest, so an update is Enter through the questions — or `.\install.ps1 -NonInteractive` with no arguments at all.

What is kept, deliberately:

- **`database.local.php`** — the JWT secret and everything else in it survive untouched; regenerating the secret would sign out every logged-in user. The connection fields (`dd_path`, `username`, `password`) are rewritten **only** if you deliberately answer differently, and only after step 4/8 has proved the new answer actually opens the dictionary.
- **Anything else you put in `backend\php\config\`** — that directory is the site's, and is never mirrored.
- **The site's `httpd.conf`** — untouched when the `Include` is already right, which after the first install it always is.
- **Apache itself** — restarted only if the rendered configuration or the `Include` actually changed. A routine application update restarts nothing.

What is *removed*: files that this release no longer ships. `frontend\dist`, `backend`, `schema`, `tools` and `docs` are mirrored, not merged, so a retired API route or a stale hashed SPA chunk from an older version does not linger — the deployed tree is exactly this bundle. `backend\php\config`, `logs\` and `apache\` are excluded from that mirroring.

What always re-runs: **the migration** (step 8/8). It is idempotent — a second run reports what was already there (`already present`) rather than doing it again — but it is not optional: a release that adds a **table** (electronic submission added `web_x12jobs`) needs it.

Then run the desktop Admin module's structure update — every time. A release that adds a **permission** delivers it that way and no other: permissions are data in `web_groups.perms`, this installer never writes them, and the structure update appends whatever the new release declares (it never removes anything a site added by hand). Reports, remittance posting and electronic submission each arrived as a new permission in September 2026. A feature gated on a permission that has not been granted is simply invisible — no error, no empty state, just a menu entry that is not there — so "the update ran but the new screen is missing" almost always means this step, not the install. Use the Admin module built for the same release as the web bundle: an older one does not know the new permission exists.

Who sees what afterwards is decided by group membership, which the site controls in the same Admin module. Not every operator is a billing user; see the table under *Accounts* before treating a missing menu entry as a fault.

If the site submits claims from the web, **update the AutoTasks service from the same release too** — see *Electronic claim submission needs AutoTasks* below. The builder and sender live there, not in this bundle.

10 Removing it

Three steps, and the installer prints them for your machine when it finishes:

1. **Delete the three marked lines** from the site's `httpd.conf` — everything from `# BEGIN Mp10 Web` to `# END Mp10 Web` inclusive — and restart that Apache service. Nothing else in that file was ever changed, so nothing else needs reverting.
2. **Drop the four tables** (`web_users`, `web_groups`, `web_group_members`, `web_x12jobs`) from the dictionary. Note that a later structure update from the desktop Admin module recreates them, empty; that is harmless.
3. **Delete the install root** (`C:\Mp10Web` by default), and `%ProgramData%\Mp10Web\` if you do not want the remembered answers kept for a future install.

Do not delete the `sequences` row for `field = 'recno'`. It is not Mp10-Web-owned data — it is shared dictionary infrastructure the desktop apps read too, and removing it (rather than merely leaving Mp10 Web's own four tables behind) would affect record-number minting outside this application entirely. Removing Mp10 Web means dropping the four `web_*` tables; it does not mean touching `sequences`.

11 Fault-finding

Every failure the installer can produce is printed with `[FAIL]` and stops the run at that point. **Before step 5/8 (Deploy files)**, that means nothing was written — whatever was there before, if anything, is exactly as it was. **From step 5/8 onward**, though, the application files under the install root have already been replaced by the time a later step can fail — a failure past that point leaves a partially-updated tree, not a rolled-back one. Treat any such failure as **incomplete, not safe**: fix the problem and re-run to a clean completion before trusting the site again. Apache's own configuration is the one deliberate exception even within that range. It cannot be validated anywhere but its final path — `httpd -t` checks the site's `httpd.conf`, which reaches `mp10web.conf` through an `Include`, so a copy staged elsewhere would only re-check content already in force. Instead the installer keeps the previous version in hand and **puts it back** if Apache rejects the new one, or if the service will not come back up — so the running site always keeps a configuration it will still start with. The table below is keyed to the exact message text; where a message embeds a value (a path, a count, a port), that part is shown as `<...>`.

> **Before you use this table, check what is actually broken.** These messages come from `install.ps1`. If the installer finished cleanly and the site loads, but **printing, signing or scanning** does not work, nothing here applies — that is the workstation side, which this installer does not touch. See The workstation side, and check the origin allowlist first.

11.1 Step 0/8 — Privileges

Message	What it means	What to do
Run this from an elevated PowerShell. It writes to the Apache configuration and restarts the Apache service.	PowerShell was not run as Administrator.	Re-open PowerShell with "Run as administrator" and re-run.
`install.ps1` cannot be loaded because running scripts is disabled on this system / is not digitally signed.	The server's execution policy refuses unsigned scripts. The installer never got as far as step 0.	`powershell -NoProfile -ExecutionPolicy Bypass -File .\install.ps1` — see *If PowerShell refuses to run it* above. If the files also prompt "Do you want to run this software?", `Get-ChildItem -Recurse | Unblock-File` in the bundle folder first.

11.2 Step 1/8 — Verify the bundle

Message	What it means	What to do
No manifest.json beside install.ps1 - is this an extracted Mp10 Web bundle?	`install.ps1` was run from somewhere other than the extracted bundle.	Run it from inside the extracted zip, next to `manifest.json`.
`<n>` file(s) missing or corrupt. Re-copy the bundle and extract it again.	Extraction was incomplete, or a file changed after extracting.	Delete the extracted folder and re-extract from a zip you have already checked against its `.sha256` sidecar.
manifest.json claims `<N>` files but lists `<M>` - the manifest itself looks corrupt. Re-copy the bundle and extract it again.	`manifest.json` itself is damaged.	Re-download the zip (checksum it first) and re-extract.

Message	What it means	What to do
This installer handles the 'update' flavour; this bundle says '<flavour>'.	The extracted bundle is not an update-flavour bundle.	Use the bundle that matches this installer; do not mix bundle types.

11.3 Step 2/8 — Preflight

Message	What it means	What to do
PHP not found at '<path>'. Install PHP 8.x ZTS x64 with php_ads, or pass -PhpPath <path to php.exe>.	No PHP at the default location or on PATH.	Install PHP 8.x ZTS x64, or pass -PhpPath.
The PHP probe produced no output. '<path>' may not be a working PHP: <raw>	The file at that path is not a functioning php.exe.	Confirm the path is really php.exe; the quoted <raw> text is whatever it actually printed.
PHP <version> found; Mp10 Web needs PHP 8.x.	PHP is the wrong major version.	Install PHP 8.
PHP is <bits>-bit; Mp10 Web needs the x64 build. (The Mp10 DESKTOP applications are 32-bit - this is a different PHP.)	**The 32-bit trap.** A 32-bit PHP was found — commonly because that is what happens to be on the machine already.	Install a separate PHP 8 **x64** build for Mp10 Web; do not point it at whatever 32-bit PHP is already there.
PHP is the NTS (non-thread-safe) build; php_ads.dll needs the ZTS build. Download the 'Thread Safe' x64 zip from windows.php.net.	Wrong PHP thread-safety variant.	Install the *Thread Safe* x64 zip, not the *Non Thread Safe* one.
The Advantage PHP extension is not loaded - AdsConnection does not exist. Copy php_ads.dll into <ext_dir> and add extension=ads to <ini>. It also needs the 64-BIT ACE client (ace64.dll, adsloc64.dll, aicu64.dll, axcws64.dll) on PATH - the 32-bit client the desktop applications install will NOT do.	php_ads is missing, disabled, or the 64-bit ACE client isn't on PATH.	Enable extension=ads in the named php.ini with php_ads.dll in the named extension directory, and put the **64-bit** ACE DLLs on PATH — not the desktop's 32-bit ones.
php-cgi.exe not found beside '<path>'. Apache runs PHP through mod_fcgid, which needs the CGI build alongside the CLI one.	php-cgi.exe is missing from the folder holding php.exe.	Copy/install php-cgi.exe into the same folder — a PHP zip normally ships both.
No Apache (httpd.exe) Windows service found on this machine. Mp10 Web mounts into the site's Apache; install Apache 2.4 as a service first (or, if it runs under another name, pass -ApacheService <name>).	No service on this machine runs httpd.exe.	Register Apache as a service (httpd.exe -k install), or name it with -ApacheService if it exists under an unexpected name.
No Apache service named '<name>'. Services found: <list>	The name given does not match any Apache service.	Use one of the names listed in the message.

Message	What it means	What to do
Service '<name>' has no configuration file registered (no -f in its ConfigArgs), so its httpd.conf can only be guessed as '<path>' …	The service was registered without -f, so Apache would fall back to that installation's default config — **which may belong to a different service.** Refused outright when running unattended; interactively it is a warning on the prompt.	Re-run with -HttpdConf naming the file this service really uses, or re-register the service with -f. Do not accept the guess unless you know that service genuinely uses the default configuration.
[**WARN**] <path> is also the configuration of: <names>. Adding Mp10 Web here affects those services too.	More than one Apache service runs this same file.	Expected if the services deliberately share a config; otherwise stop and check which one should carry Mp10 Web.
Service '<name>' points at '<exe>', which does not exist.	The service is registered against an httpd.exe that has been moved or deleted.	Repair or re-register that Apache installation before installing Mp10 Web.
httpd.conf not found at '<path>'.	The configuration path given (or derived from the service) is not there.	Pass the right file with -HttpdConf.
Apache at '<server root>' is missing: <mod_rewrite.so, mod_fcgid.so>. …	That Apache's modules\ folder is missing one or both files.	Install a full Apache 2.4 build (e.g. Apache Lounge) that ships mod_rewrite.so and mod_fcgid.so — a trimmed-down build will not do.

11.4 Step 3/8 — Configuration

Message	What it means	What to do
Install root must be a local path like C:\Mp10Web, not a UNC or relative path.	A UNC or relative install root was given.	Use a real local path.
<root> is on a mapped network drive (<target>). Apache cannot see it.	The install root is a mapped drive; a service running as LocalSystem cannot resolve one.	Install to a real local disk.
A dictionary path is required (-DdPath, or run interactively).	The "Path to mp.add" prompt was left blank, or -NonInteractive was used with nothing remembered.	Supply the real path to mp.add.
An ADS username is required.	The username prompt was left blank.	Supply an ADS username.
ADS password is required and this session cannot prompt for one. Run interactively once; later runs reuse it.	Running unattended with no saved password to fall back on.	Install once interactively so the password is on file in database.local.php, or pass -AdsPassword for that first run only.
URL path '<path>' is not usable. Use letters, digits, . _ - and / - e.g. /mpweb or /apps/mp10.	The URL path answer contains characters that cannot be an Apache alias, or was bare /.	Answer with a path like /mpweb.
Public origin must be scheme://host[:port] with no path - got '<origin>'.	A full URL with a path or query was given.	Answer with just the scheme and host, e.g. https://mp10.clinic.local.

Message	What it means	What to do
[**WARN**] The origin's port (`<n>`) is not among the ports this Apache listens on (`<list>`).	The origin does not match any `Listen` in that Apache's configuration.	Fine behind a reverse proxy or load balancer. Otherwise the origin is wrong — staff would type an address nothing answers on.

11.5 Step 4/8 — Test the dictionary connection

Message	What it means	What to do
The connection test produced no result. Check that php_ads is loadable.	PHP failed to run the connection probe at all.	Run `php -m` and confirm `ads` loads without error.
Could not open the dictionary: `<ADS error>`	The path, credentials, or network path to `mp.add` is wrong, or the server is unreachable.	Read the quoted ADS error — it states exactly which of those it was. Password is redacted from this message even on a credential failure.

11.6 Step 5/8 — Deploy files

Message	What it means	What to do
robocopy failed copying '`<dir>`' to `<root>` (exit code `<code>`). Check permissions on the destination.	The install root could not be written to (permissions, a locked file, antivirus).	Check NTFS permissions on the install root and that nothing has a file there open/locked. Directories copied earlier in this same step may have already succeeded before this one failed, so the install root is now a MIXED, incomplete tree — do not point Apache at it or consider the site trustworthy until you fix the problem and re-run to a clean completion.

Step 6/8 (Configuration file) has no row here on purpose: writing a fresh `database.local.php`, or keeping an existing one, has no distinct failure message to catch — nothing was skipped.

11.7 Step 7/8 — Apache

Message	What it means	What to do
httpd.conf already Includes an mp10web.conf outside the Mp10 Web markers. Remove that line and re-run, so the installer can manage it.	Somebody added the `Include` by hand, without the `# BEGIN/# END` markers. Adding a second one would load the same aliases twice.	Delete the hand-written `Include` line and re-run; the installer then adds and owns a marked one.

Message	What it means	What to do
Apache rejected the configuration - see above. The site's Apache is unchanged, but the application files under `<root>` were already updated. Fix the problem and re-run to a clean completion.	`httpd -t` refused the site's configuration with Mp10 Web's `Include` in it. **The `httpd.conf` backup has already been restored at this point** — the running site is safe.	Read the `httpd -t` output printed just above. Note it may name a problem that was *already* in that `httpd.conf` and simply never re-validated since the last restart — that is common, and it is not caused by Mp10 Web. Fix it, then re-run.
Apache service '`<name>`' would not `<verb>` with Mp10 Web's Include, but started again once it was removed …	Apache refused to come back up, and starting it without the Include worked. **`httpd.conf` has already been restored and the site is back up — without Mp10 Web.**	Read Apache's `ErrorLog`: something the Include loads (a module, a path) is unacceptable to this Apache at runtime even though the syntax check passed.
Apache service '`<name>`' is not running after `<verb>`, and did not start with Mp10 Web's changes reverted either …	The service could not run before this install either.	Nothing here is Mp10 Web's doing. Check Apache's `ErrorLog` and whether the service is properly registered (see any ConfigArgs warning earlier in the run).

11.8 Step 8/8 — Database

Message	What it means	What to do
The database step failed - see above.	`tools\migrate.php` exited with an error, or printed a real (uppercase) `FAILED` line.	Read the migration output printed above — it names the specific step that failed.
WARNING: sequence=`<n>`, but the next number (`<recno>`) is already taken…	`sequences.recno` has fallen behind the patient record numbers actually in use. Patient creation still works — it probes for a free number — but it is retrying past taken ones, and enough of them in a row will exhaust its 50 attempts.	Investigate in Admin10 before creating patients. **Do not "fix" it by raising the counter blind:** that row is shared with the desktop applications, which mint patient numbers from it too, and `MAX(recno)` is not a reliable target — one mistyped record number sitting above the real block would burn every number in between, irreversibly.

11.9 Smoke test

The smoke test posts a login for a randomly generated username to `<prefix>/api/auth/login` on `127.0.0.1`, and requires **401** back. That one answer proves the whole chain end to end — Apache matched the alias, the `.htaccess` rewrite ran, `mod_fcgid` started PHP, `php_ads` loaded, and the dictionary answered a query against `web_users` — because the login handler has to read the account table before it can refuse.

Message	What it means	What to do
Login probe at `<url>` returned `<code>`; 401 was expected. PHP or the Apache mount is not right - see above.	Something answered, but not correctly. A **500** usually means PHP ran and failed (check `database.local.php` and the ACE client); a **404** means the alias did not match; a **200** would mean an unauthenticated login succeeded, which is serious.	The response body and the tail of Apache's `ErrorLog` are printed just above the message — read those first.
Nothing answered `<prefix>`/api/auth/login on 127.0.0.1 ports `<list>` within 20s (`<err>`). Check the '`<name>`' service.	Nothing was listening on any port that Apache's configuration says it listens on.	Confirm the service is running, and that the ports in its configuration are the ones actually bound.